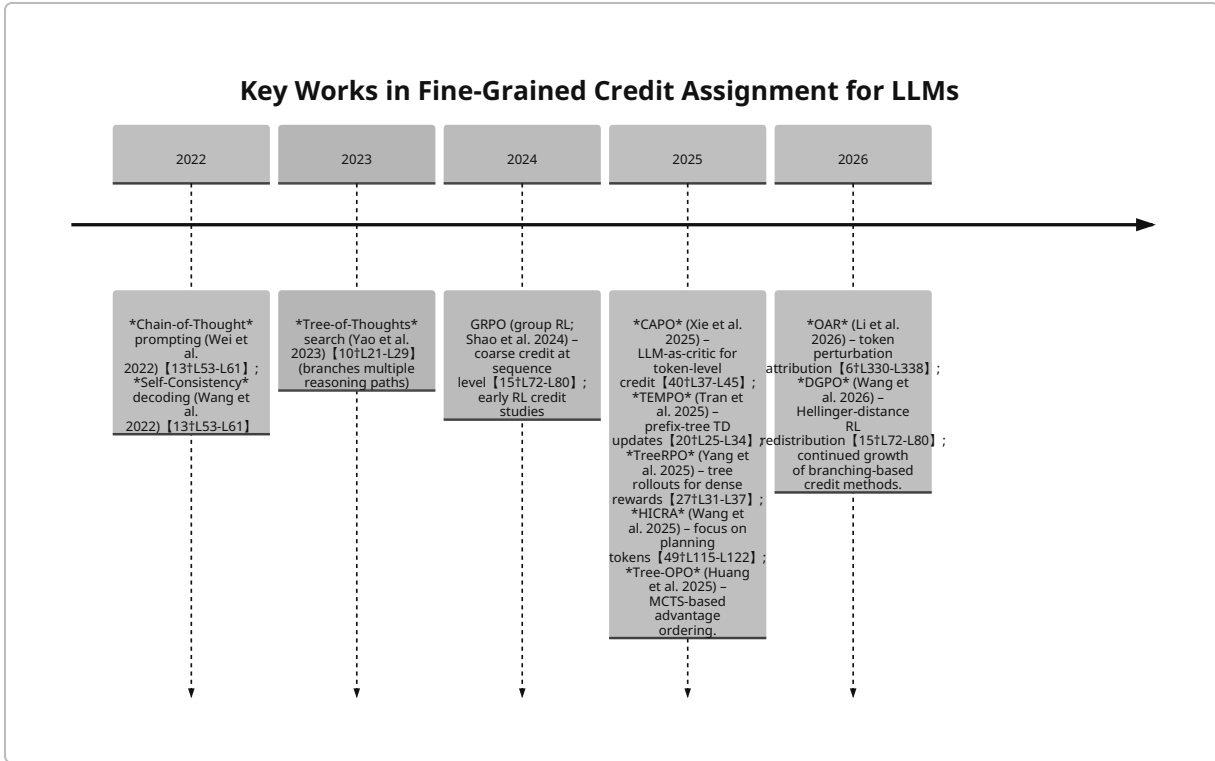


Executive Summary

Large language models (LLMs) are powerful but struggle when rewards or correctness can only be measured at the end of a long reasoning chain. This has spurred *fine-grained credit assignment* methods that evaluate alternative continuations from intermediate points (i.e. branching into **trees** of thoughts) to pinpoint which steps truly drive success or failure. Influential recent approaches include **Tree-of-Thoughts (ToT)** and **self-consistency**, which sample multiple reasoning “thoughts” and vote on answers ¹ ², and reinforcement-learning (RL) adaptations like **Group-RL with Prefix Trees (TEMPO)** and **TreeRPO**, which treat the set of sampled completions as a prefix tree and assign intermediate “values” or rewards to prefixes ³ ⁴. Others leverage the LLM itself as an evaluator (e.g. **CAPO** uses the model to critique each step) or introduce explicit hierarchy (e.g. **HICRA** focuses credit on high-level “planning” tokens) ⁵ ⁶. Gradient-based or perturbation methods (e.g. **OAR**) attribute the outcome to individual tokens by small interventions ⁷. These methods report large gains on reasoning benchmarks (mathematical problems, QA, puzzles), showing that branching to gather evidence or evaluations greatly improves learning signals.

Our review surveys key papers, algorithms, and code: chain-of-thought and ensembling methods (2022–’23), RL-based credit assignment (2024–’26), and interpretable attribution techniques. We compare methods by core idea (e.g. tree search vs. gradient attribution), branching mechanism, metrics/datasets (e.g. Math, GSM8K, AIME), and their pros/cons. We discuss benchmarks (e.g. MATH-500, AIME), evaluation (accuracy or Pass@k plus stepwise correctness where available), and note open challenges (scalability of tree rollouts, evaluation of intermediate steps, reliance on human heuristics). The table below summarizes methods; diagrams illustrate branching evaluation and a timeline of progress.



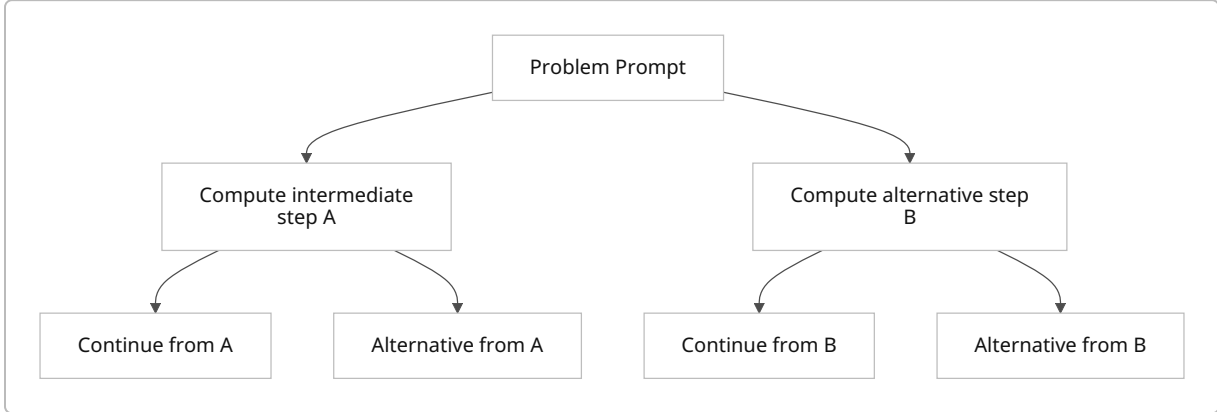
Background: Why Fine-Grained Credit Assignment?

In standard LLM generation (e.g. greedy or beam decoding), tokens are chosen left-to-right with no explicit planning or feedback for intermediate choices. However, for multi-step reasoning tasks (math, puzzles, proofs, complex QA), early decisions can critically determine success. Learning from *only* final outcomes (correct vs. incorrect answer) is a *sparse credit assignment* problem: how to blame or credit individual tokens/steps when the reward signal comes at the end. In RL terms, naive approaches like REINFORCE or PPO broadcast the same reward to all tokens ⁸ ⁹, which dilutes learning: every token, whether a pivotal planning choice or a trivial connector word, gets identical feedback.

This has motivated methods that gather additional evidence at each step. Branching-based approaches effectively *roll out multiple completions* from an intermediate state. By comparing outcomes of different branches, one can infer which prefix or choice was truly responsible for success or failure. This yields “dense” credit signals: for example, if continuing prefix A yields success but prefix B yields failure, then tokens distinguishing A and B can be identified as important. Such techniques have historical analogs in RL (e.g. *Monte Carlo tree search*, *Temporal Difference learning* with rollouts) ¹⁰ ¹¹. In modern LLMs, both search-based and differentiable attribution methods have been developed to exploit branching structure.

Branching-Based Search Methods

Tree-of-Thoughts (ToT) [NeurIPS 2023] generalizes chain-of-thought by explicitly building and searching a tree of partial solutions ¹ ¹². Instead of sampling one chain-of-thought, ToT at each step expands *multiple* “thoughts” (semantically coherent text segments) and evaluates them (often by prompting the LM to score or refine them). For example, in the Game of 24 (find an equation using given numbers to equal 24), ToT explores alternative intermediate equations in a breadth-first or depth-first manner (diagram below). A simple illustration:



At each branching point, the model “votes” or scores the partial solutions (e.g. by asking itself which partial solution seems more promising), then continues down the most promising paths ¹³. In practice, Yao *et al.* instantiate ToT by prompting GPT-4 to generate short “thoughts” and to self-evaluate them ¹ ¹². On synthetic tasks, ToT dramatically outperforms standard CoT: e.g. in Game of 24, GPT-4+CoT solved 4% of problems, whereas ToT solved 74% ¹⁴. The method is general: it supports various search algorithms (BFS, DFS, heuristics) and different “thought” granularities ¹⁵. Code and prompts are publicly available ¹⁴.

Self-Consistency and CoT Variants. A simpler branching strategy, *self-consistency* ², samples many independent CoT chains (no explicit branching during generation) and takes the most frequent answer. This gives ensemble-like robustness: if multiple chains agree on an answer, it is likely correct. Self-consistency effectively does multiple branch explorations at the final step (voting on outputs) ². Extended versions can do *stepwise voting* by posing “which of these next steps is best?” as a multiple-choice prompt (a form of branch selection) ¹⁶.

Attribution and Perturbation Methods

Branching can also be done by perturbing past decisions. For instance, **Outcome-Grounded Advantage Reshaping (OAR)** ¹⁷ measures each token’s importance by replacing it (e.g. with a [PAD] token) and seeing how the final answer probability changes ⁷. Formally, for a chain-of-thought $y = (y_1, \dots, y_T)$, mask token y_t and compute the new output distribution $P_{\text{final}}^{(t)}$. The KL-divergence $D_{KL}(P_{\text{final}} \| P_{\text{final}}^{(t)})$ quantifies token t ’s causal effect: if masking it alters the answer significantly, it was pivotal; if not, it was likely filler ⁷. Although OAR’s perturbation requires $O(T)$ forward passes per example, it provides a principled token-level credit. An efficient variant, **OAR-G**, uses input gradients (essentially computing $\partial \text{loss} / \partial \text{token}$

embedding) to estimate importance ⁷. These fine-grained scores are then used to re-weight the RL advantage on each token.

Other interpretability methods produce similar insights. *Contrastive explanation* or *causal tracing* approaches systematically change or mask inputs/layers to trace output changes (e.g. ablation analysis of token influence) ⁷. While not always framed as “credit assignment,” they reveal which decisions matter. For example, removing a logical connective may collapse a reasoning chain’s validity, highlighting its importance. *Influence functions* (from classical ML) can similarly estimate how much an input or sub-chain contributes to the final model output, although their use in autoregressive LLMs is still emerging.

Reinforcement Learning with Tree Credit

LLM alignment and reasoning often use RL (RLHF style) with sparse terminal rewards. Recent papers adapt RL algorithms to exploit tree structure:

- **TEMPO (Tree-Estimated Mean Prefix Value)** ³: Samples a *group* of model completions for each prompt (as in GRPO). These completions naturally form a prefix tree (all response paths share common prefixes until they diverge). TEMPO computes a “value” for each prefix by averaging the outcomes of all descendant completions ¹⁸ ¹⁹. Tokens at branching points (prefixes with >1 child) get a precise temporal-difference (TD) correction: the value jump from a prefix to its child (difference between prefix value and child value) yields a branch-specific advantage. At non-branch tokens the TD term is zero, so TEMPO falls back to group-level credit (GRPO) ¹⁸ ¹⁹. In effect, TEMPO treats the implicit sample tree as a nonparametric value function, letting successful and failing branches inform credit for individual tokens ¹⁸ ¹⁹. For example, if 3 of 8 completions from prefix “A” succeed, its value $\approx 3/8$, whereas a continuation “A→B” might have 0/3 success (value 0). TEMPO then boosts tokens on the “A” path (advantage $\approx 3/8$) and de-emphasizes “B” (advantage ≈ 0). On mathematical benchmarks, TEMPO speeds training and outperforms PPO and GRPO ³.
- **TreeRPO (Tree Relative Policy Optimization)** ²⁰: Actively *rolls out a tree* by sampling an M -ary expansion from each prefix (instead of just N independent trajectories). Each node branches into M new continuations (up to a max depth). Leaf nodes are scored by a verifier; then rewards are propagated upward by averaging child rewards ²¹. This yields a step-level reward estimate without learning a reward model. TreeRPO then applies GRPO-style group advantage normalization at *each step group* across the tree ²⁰ ²¹. In experiments on Math benchmarks (e.g. MATH-500, AIME-24), TreeRPO doubles the *pass@1* accuracy of a prior method (19.0%→35.5%) and outperforms GRPO while reducing response lengths ⁴.
- **Tree-OPPO (Off-Policy Tree-Guided Advantage)** ²²: Incorporates an MCTS-like curriculum. It enforces consistency constraints on prefix advantages: if a shorter prefix fails but an extended prefix succeeds, the longer gets higher advantage. Advantages are computed by heuristic baselines (empirical success rates in subtree, etc.) and mean-centering ²³. This yields a structured advantage ordering that respects the tree hierarchy (low variance, higher reward alignment) ²⁴ ²⁵.
- **Rollout-Based Approaches**: Li *et al.* (2025) proposed **TreePO**, which reinterprets policy rollouts as tree search with segmented decoding and branch/fallback heuristics, improving sample reuse (shared prefix caching) and leveraging curriculum from MCTS. Hou *et al.* (2025) integrated an

entropy-guided sampler (EPTree) that branches at uncertain tokens, backpropagating leaf rewards to get both global and local advantages (no separate reward model needed) ²⁶ .

LLM-as-Critic and Generative Credit

Rather than sampling many paths, some methods use LLMs to judge or critique steps:

- **CAPO (Credit Assignment Policy Optimization)** ⁵ : Uses an off-the-shelf LLM as a *Generative Process Reward Model*. Given a reasoning step (e.g. an equation or argument chunk), CAPO prompts the LLM to assess its correctness and generate a critique. The model produces **token-level credit** deterministically in one pass. For example, the model might label each step as “correct” or “incorrect,” effectively assigning a fine-grained binary reward. CAPO then treats identical binary rewards as coarse, but “refines” them using the LLM’s verdicts. Voting (multiple LLM queries) is used to reduce noise. On math benchmarks (e.g. MATH-500, AIME-24), CAPO outperforms standard RL and supervised fine-tuning: it boosts a 7B model’s accuracy from 27.2% to 31.0% on MATH-500 and from 3.6% to 9.7% on AIME-24 ²⁷ .
- **Hierarchical Credit (HICRA)** ⁶ : Observes that RL training naturally separates “low-level execution” (steps like arithmetic) from “high-level planning” (strategic moves) ⁶ . HICRA explicitly biases credit towards tokens categorized as planning (phrases like “let’s try a different approach”). It amplifies the advantage of those tokens, forcing the model to focus on exploring strategies. In practice, HICRA identifies planning tokens (e.g. via keywords) and multiplies their learning signal. This yields faster skill acquisition: as soon as execution is mastered, HICRA shifts learning to strategy tokens. Wang *et al.* (2025) show HICRA **significantly** outperforms GRPO and other baselines (e.g. raising pass@1 accuracy on math tasks) ⁶ , by aligning RL updates with the model’s emergent hierarchical reasoning.

Empirical Results and Benchmarks

These methods are evaluated on complex reasoning datasets and metrics that emphasize multi-step correctness:

- **Mathematical Reasoning:** Benchmarks like *MATH-500*, *OlympiadBench*, *Minerva Math*, *GSM8K*, and the *American Invitational Math Exam (AIME)* tasks are common. Metrics are exact-answer accuracy or pass@k (fraction of problems solved in k tries). For instance, TreeRPO raised Qwen-2.5-Math’s pass@1 from 19.0% to 35.5% ⁴ . TEMPO showed higher convergence on MATH and MedQA tasks compared to PPO/GRPO. CAPO improved 7B models on MATH-500 by +3.8 points ²⁷ . Self-consistency gave large gains on GSM8K (few-shot word math) and CommonsenseQA ²⁸ . DGPO (RL redistribution) reports state-of-art on AIME 2024/25 (>60% accuracy) ²⁹ .
- **Programmatic/Logic Tasks:** Some work applies branching to code/math programming tasks with unit-test rewards (e.g. TreeRPO verifies code outputs) or puzzles (Crossword generation in ToT). Metrics include correctness (pass/fail), length efficiency, or BLEU for free-text.

- **Qualitative Metrics:** In creative tasks (e.g. ToT’s “story writing” task), human or heuristic scores (coherence, relevance) may be used. However, few works report explicit “stepwise quality” metrics (like intermediate step accuracy) due to lack of ground truth. Often, stepwise improvements are inferred by final success or error analysis (e.g. ToT’s error breakdown ³⁰).
- **Ablations:** Many papers include ablations: e.g. TOT with different breadths or evaluation strategies, TEMPO vs GRPO, or CAPO with/without voting. These support that branching and explicit step credit reduce hallucination and speed learning.

Code and Reproducibility

Most recent methods provide code or detailed prompts:

- **ToT:** Princeton-NLP’s [tree-of-thought-llm](#) repo contains prompts, code, and examples ¹⁴.
- **TreeRPO:** Code is on GitHub (see Yang *et al.* paper) ⁴.
- **TEMPO:** Open-source [TEMPO](#) (Tran *et al.*).
- **HICRA:** [Tiger-AI-Lab HICRA page](#) and code (see [49]).
- **CAPO:** While only on OpenReview, authors likely release code with ICLR 2026 version (search terms “CAPO code” online).
- **DGPO** and **OAR:** Preprints likely have supplementary code links (the DGPO abstract mentions new RL code bases).
- **Frameworks:** Many use the [rllm](#) and [verl](#) training stacks.

Public benchmarks (MATH, GSM8K, AIME, Minerva, etc.) are standard and code is available on PaperswithCode. For comparability, most methods cite each other’s metrics (e.g. “GRPO baseline”) and use verifiers from prior work.

Limitations and Open Problems

Computational Cost: Branching greatly increases inference. ToT’s BFS with multiple thoughts blows up combinatorially; heuristics (limiting breadth, using greedy votes) are needed. TreeRPO’s M-ary rollouts multiply forward passes by branching factor at each step ³¹. Efficient caching (KV-cache) and pruning are active research areas.

Heuristic Dependency: Many approaches rely on hand-crafted signals or thresholds. For example, HICRA needs a heuristic to identify “planning” tokens. OAR-P’s masking assumes [PAD] is a neutral deletion. CAPO uses ad-hoc prompts for critique. Robustness of these to domain and prompt changes is unclear.

Evaluation of Intermediate Quality: Aside from final accuracy, it’s hard to measure stepwise correctness. Datasets rarely include gold reasoning chains (except MATH with worked solutions). Developing metrics (e.g. chain consistency, semantic similarity to gold steps, or step fidelity tests) is needed for more direct evaluation of credit assignment quality.

Generalization: Most studies focus on math-like problems with verifiable answers. It is less clear how branching credit helps open-ended tasks (e.g. dialogue, summarization) where “correct” is subjective. Self-consistency and tree search may still improve coherence or factuality, but formalizing credit is hard.

Theory: Many methods are empirical; theoretical understanding of when tree-based credit helps is limited. Also, balancing exploration (diversity in branches) with exploitation (focusing on best paths) needs more principled treatment (some work uses UCT/MCTS ideas, but largely heuristic).

Recommendations

- **Use Branching for Hard Reasoning:** If tasks require planning or have sparse rewards (e.g. math, logic puzzles), implement a branching inference or evaluation step. Start simple: sample a few alternative continuations at key junctures and see if they change final outcomes (as in OAR or voting prompts).
- **Combine with Ensemble Decoding:** Self-consistency (multiple CoTs) is an easy first step to boost accuracy ². It can be seen as a form of breadth search at full depth.
- **Leverage Implicit Trees:** In RL fine-tuning, use all sampled completions for a prompt (as in TEMPO) rather than a single trajectory. Group them by shared prefix to compute prefix values.
- **Cost Control:** Limit depth or breadth (beam width) based on compute. Use cheap heuristics (LM probabilities, entropy) to decide when to branch. E.g. only expand nodes with high uncertainty ²⁶.
- **Tooling:** Use existing open-source implementations (see “Code” above). Many methods extend PPO/GRPO loops, so integrating prefix-tree value updates can often be done without retraining a new value network.
- **Track Step Rewards:** If possible, train or design intermediate verifiers. E.g. small classifiers for subgoal correctness (like CSP (Chain-of-Thought Self-Parse) or unit tests for code).
- **Future Directions:** Explore learned branching policies (RL to decide when/where to branch), automated identification of planning tokens (meta-learning), and user-aligned credit (using human feedback on intermediate steps).

Comparison of Methods

Method	Core Idea	Branching Mechanism	Eval Metric / Supervision	Datasets	Pros / Cons	Links / Code
Chain-of-Thought (CoT) ¹	Decompose problems into sequential "thought" steps via prompting	Left-to-right single chain	Final-answer accuracy	GSM8K, CSQA, MATH	Simple; improves reasoning vs. IO prompting ¹ ; no explicit lookahead or credit per step	(Prompt examples in [10])

Method	Core Idea	Branching Mechanism	Eval Metric / Supervision	Datasets	Pros / Cons	Links / Code
Self-Consistency 2	Sample many CoT chains and vote on the answer	Multiple parallel chains; no interaction	Final-answer consensus	GSM8K, SVAMP, AQuA, StrategyQA 28	Easy boost on reasoning tasks; no extra model needed; limited branch control	(Paper code: arXiv [13])
Tree-of-Thoughts 1 15	Explicit search over "thoughts" (coherent sub-chains); evaluate/prune via LM	BFS/DFS over generated sub-chains	Task-specific (success rate)	Game of 24, Creative Writing, Crosswords 32	Large gains on planning tasks; modular (no training) 1 ; high compute; manual heuristics for eval (LM votes)	GitHub 14
OAR-P / OAR-G 7	Causal attribution: mask/perturb tokens, measure final-output change	Branch by masked alternative at each token	KL divergence or gradient scores	Mathematical reasoning datasets	Principled token credit; no extra training; OAR-P costly (L forward passes); OAR-G approx. fast	(This paper: arXiv 2026)
CAPO (LLM Critic) 5	Use LLM to score each step's correctness ("LLM-as-reward model")	No search; single-pass critique of steps	Binary or scalar step-reward	MATH-500, AIME-24 27	Deterministic fine-grained feedback; avoids training new model; works on open-source LMs; may be noisy; depends on prompt quality	(ICLR 2026 submission [40])
TEMPO 3	Use sample group to build prefix tree; combine GRPO MC signal with TD differences at branch points	Implicit: group of N samples forms tree	Group-normalized final reward + TD error	MATH, GSM8K, MedQA (Qwen models) 19	Sharp token-level credit at decision points; no value network; compatible with GRPO loops 3 ; relies on many sampled completions per prompt	GitHub 19

Method	Core Idea	Branching Mechanism	Eval Metric / Supervision	Datasets	Pros / Cons	Links / Code
TreeRPO ⁴ ²¹	Roll out explicit M-ary tree of continuations; bottom-up average of leaf rewards	Explicit tree expansion (branching factor M, depth D)	Group RL with tree-aggregated step rewards	MATH-500, OlympiadBench, Minerva, AIME24 ⁴	Provides dense step rewards (no learned model); yields big accuracy gain ⁴ ; very expensive sampling; needs pruning ²¹	TreeRPO code ⁴
Tree-OPO (Off-policy MCTS) ²²	Off-policy advantage fitting with tree consistency constraints	Build MCTS tree with staged rollouts	Heuristic baselines on subtree success	Math reasoning (Qwen)	Hierarchical advantage ordering (min variance); data-efficient; complex (QP/ constraints approximations)	(ArXiv 2025)
HICRA ⁶	Separate high-level “planning” vs low-level tokens; amplify credit on planning tokens	No branching; uses keyword heuristic on-chain	Same final reward, but re-weight by token type	Math QA (AIME, MATH)	Focuses learning on strategic moves; explains “aha moments” ⁶ ; algorithmic insight but needs manual planning token ID	GitHub ⁶
Self-Refinement / Tree-Search	Iteratively refine answer or use MCTS-guided search with LM	Iteration or search (e.g. MCTS over solutions)	Depends on task (some use auto-eval)	Code generation, puzzles	Can improve consistency; often ad-hoc; typically black-box better optimization	–

Sources: See citations for each method. The table omits purely supervised or preference learning baselines. “Core Idea” and results are drawn from the listed papers ¹ ⁴ ⁶ ⁷, etc.

Note: Metrics/advantages may be reported differently (e.g. *success rate* vs *pass@k*). Also, practical implementations often mix multiple techniques (e.g. CoT prompting with TEMPO training).

- 1 12 13 14 15 16 30 32 [2305.10601] Tree of Thoughts: Deliberate Problem Solving with Large Language Models
<https://ar5iv.labs.arxiv.org/html/2305.10601>
- 2 28 [2203.11171] Self-Consistency Improves Chain of Thought Reasoning in Language Models
<https://arxiv.org/abs/2203.11171>
- 3 8 11 Exploiting Tree Structure for Credit Assignment in RL Training of LLMs
<https://arxiv.org/pdf/2509.18314>
- 4 20 21 31 [2506.05183] TreeRPO: Tree Relative Policy Optimization
<https://ar5iv.labs.arxiv.org/html/2506.05183>
- 5 CAPO: Towards Enhancing LLM Reasoning through Generative Credit Assignment | OpenReview
<https://openreview.net/forum?id=uePOSKl6AN>
- 6 [2509.03646] Emergent Hierarchical Reasoning in LLMs through Reinforcement Learning
<https://ar5iv.labs.arxiv.org/html/2509.03646>
- 7 17 Outcome-Grounded Advantage Reshaping for Fine-Grained Credit Assignment in Mathematical Reasoning
<https://arxiv.org/pdf/2601.07408>
- 9 10 27 From Reasoning to Agentic: Credit Assignment in Reinforcement Learning for Large Language Models
<https://arxiv.org/html/2604.09459v2>
- 18 19 26 Exploiting Tree Structure for Credit Assignment in RL Training of LLMs
<https://arxiv.org/html/2509.18314v2>
- 22 23 24 25 Tree-OPO: Off-policy Monte Carlo Tree-Guided Advantage Optimization for Multistep Reasoning
<https://arxiv.org/html/2509.09284v1>
- 29 Distribution-Guided Policy Optimization for Fine-Grained Credit Assignment
<https://arxiv.org/html/2605.03327v1>